

Analyse détaillée de la flotte — supervision

Généré le 2026-07-24 11:22 — étage déterministe (scan 0 token) + étage qualitatif (audit-technique, lecture du code).

VSCode — Bac à sable proto PPT (COMOP, Node.js)

Dernier commit : 2026-07-24T10:05:45+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	moyen	1 fichier(s) de test, pas de coverage
Test fonct.	moyen	1 test(s) à vérification réelle
Revue code	ok	hook pré-commit, bmad-code-review
Revue incr.	ok	skill + hook SessionStart
Design	moyen	deck-design-library, ppt-designer
Doc	moyen	wiki, CLAUDE.md
Cadrage produit	moyen	besoins + brief BMAD
Pratiques+rules	moyen	CLAUDE.md
Sécu (proxy)	ok	.env gitigné, deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ok

Commentaire : Gestion d'erreurs globalement solide (validation Mandatory + Test-Path côté PowerShell, garde-fous de taille et de chemin côté Node), avec quelques incohérences mineures de codes d'erreur.

EXEMPLE CONSTATÉ	LOCALISATION
JSON malformé -> 500 générique au lieu de 400 : JSON.parse non entouré d'un try/catch local	<code>server.js:303,265</code>
Incohérence : safeTemplatePath try/catché en 400 ailleurs mais pas dans /api/generate (template invalide -> 500)	<code>server.js:304</code>

Générateur PS bien gardé : paramètres Mandatory, Test-Path sur TemplatePath/DataPath, nettoyage workDir en finally

src/generate-comop.ps1:1-12, 34-39, 86-89

process.exit(1) sur uncaughtException : un crash isolé tue tout le serveur (pas de dégradation gracieuse)

server.js:376-384

performance — ● ok

Commentaire : Aucun enjeu réel pour un prototype local : un process PowerShell par requête et zip extrait/recompressé à chaque appel, acceptable à cette volumétrie.

EXEMPLE CONSTATÉ	LOCALISATION
Spawn d'un powershell.exe complet + extraction/recompression zip du template à chaque génération : coûteux mais non critique à cette échelle	server.js:88-106, 317-326

risque_technique — ● moyen

Commentaire : Proto assumé mais fragilisé par l'absence de package.json malgré Node, l'absence de CI et de tests unitaires (un seul smoke-test), et de la duplication de fonctions entre scripts PowerShell.

EXEMPLE CONSTATÉ	LOCALISATION
Aucun package.json malgré un serveur Node : pas de manifeste ni version de moteur déclarée (stdlib uniquement, zéro dépendance)	comop-pptx-prototype/ (racine)
Couverture limitée à un seul smoke-test.ps1 (14 assertions e2e), aucun test unitaire ni CI	src/smoke-test.ps1
Duplication de helpers (Get-ShapeName, Get-GraphicType, New-TempDirectory) copiés entre scripts	src/detect-template-zones.ps1:10-51 vs remove-template-shape.ps1:14-27
Manipulation du PPTX par regex sur le XML plutôt qu'un parseur : fragile face aux variations OOXML	src/generate-comop.ps1:61-66
remove-template-shape écrase le template source (opération destructive sans sauvegarde)	src/remove-template-shape.ps1:74-77

securite — ● moyen

Commentaire : Le risque n°1 (injection de commande Node→PowerShell) est bien traité (spawn en tableau d'arguments, chemins assainis, XML échappé), mais le serveur écoute sur toutes les interfaces (0.0.0.0) alors qu'il exécute PowerShell et accepte des templates uploadés — exposition LAN non voulue.

EXEMPLE CONSTATÉ	LOCALISATION
Serveur exposé sur toutes les interfaces : server.listen(port) sans hôte -> écoute 0.0.0.0 ; le service qui exécute PowerShell et accepte l'upload de templates est accessible depuis le LAN (le README parle de localhost)	server.js:386

Invocation sûre : spawn('powershell.exe', [args]) en tableau, sans shell:true ni interpolation -> pas d'injection de commande	server.js:88-106
Path traversal gardé : safeTemplatePath impose basename+.pptx+startsWith(templatesDir) ; serveStatic vérifie startsWith(webDir)	server.js:108-115,354-355
Données utilisateur échappées (SecurityElement::Escape) avant insertion XML -> pas d'injection OOXML	src/generate-comop.ps1:16-26
ExecutionPolicy Bypass systématique + extraction de zip fourni par l'utilisateur : surface résiduelle (zip-slip théorique selon version .NET)	server.js:90 ; src/generate-comop.ps1:47

VSCode1 — Questionnaire maturité agile/produit + export PPT

Dernier commit : 2026-07-24T10:13:04+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	moyen	11 fichier(s) de test, pas de coverage
Test fonct.	moyen	1 test(s) à vérification réelle
Revue code	ok	agent reviewer, hook pré-commit, bmad-code-review
Revue incr.	ok	skill + hook SessionStart
Design	ok	deck-design-review, deck-design-library, ppt-designer
Doc	ok	README+usage, wiki+html, CLAUDE.md
Cadrage produit	moyen	persona, why
Pratiques+rules	ok	linter, CI, CLAUDE.md, conventions
Sécu (proxy)	ok	deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ok

Commentaire : Validation des entrées systématique (types, champs requis, périmètre de session) et ré-import référentiel transactionnel/idempotent bien maîtrisé, à quelques atomicités et échecs silencieux mineurs près.

EXEMPLE CONSTATÉ	LOCALISATION
------------------	--------------

replacerInvites fait DELETE puis INSERT en boucle hors transaction : un échec en cours laisse les invités partiellement remplacés (non atomique, contrairement à reconcileReferentiel)	app/src/invites.js:59-65
Nettoyage des fichiers temp d'export silencieux : unlink().catch(()=>{}) ignore l'échec, fuite possible dans os.tmpdir()	app/src/server.js:868-871
sessionStatus compare des getTime() non gardés : une date stockée invalide donnerait NaN et un statut 'ouverte' par défaut sans erreur	app/src/server.js:42-49

performance — ● moyen

Commentaire : Agrégation des scores en N+1 (une requête SQL par question) ré-exécutée plusieurs fois par export, acceptable sur la faible volumétrie d'un questionnaire mais structurellement coûteuse.

EXEMPLE CONSTATÉ	LOCALISATION
N+1 : une requête 'SELECT ... FROM reponses WHERE question_id=?' par question, imbriquée dans les boucles pilier→sous-catégorie→question	app/src/server.js:541-544 (agregerResultats)
Recherche linéaire imbriquée repondants.find() : O(réponses × répondants) par question	app/src/server.js:547
Ré-agrégation redondante à l'export département (2× agregerResultats, ~3×(N+1) fois pour N équipes)	app/src/server.js:708-803, 637-651
Aucune pagination sur les listings (sessions, valeurs, résultats) — impact limité par la volumétrie	app/src/server.js:109-114, 166-173

risque_technique — ● moyen

Commentaire : Le cœur de calcul (agrégation de scores, moyenne, écart-type, comparaison historique) n'a aucun test unitaire JS et le seul test du chemin export n'est pas branché dans npm test.

EXEMPLE CONSTATÉ	LOCALISATION
Chemin critique de calcul des scores non testé : agregerResultats/calculerComparaison/moyenne/ecartType absents de la suite npm test	app/src/server.js:511-695 vs package.json 'test'
test-export-ppt.py (seul test du livrable PPT) non inclus dans npm test qui n'enchaîne que des .js — le livrable principal échappe à la commande standard	app/scripts/test-export-ppt.py ; package.json: 'test'
Duplication quasi identique des routes /equipes et /departements et du motif manager==='sans' répété ~6 fois	app/src/server.js:432-466, 532, 592, 622
Couplage fort au singleton db importé directement par tous les modules — testabilité réduite	app/src/referentiel.js:2, invites.js:2, server.js:9

securite —  moyen

Commentaire : SQL entièrement paramétré, pas de shell ni d'eval et aucun secret en clair, mais l'API — qui expose des données nominatives (nom/prénom/email) — n'a strictement aucune authentification.

EXEMPLE CONSTATÉ	LOCALISATION
AUCUNE authentification/autorisation sur toute l'API : n'importe qui atteignant le serveur peut lister les répondants avec PII, créer/supprimer/fusionner, exporter (grep auth/session/token = vide)	app/src/server.js (toutes les routes /api/*)
SQL sain : requêtes paramétrées partout ; interpolations = liste blanche CHAMPS_FUSIONNABLES + placeholders générés — pas d'injection	app/src/server.js:164-186,776 ; referentiel.js:276-286
.env.dev/.preprod/.prod committés alors que .gitignore ne les ignore pas (ici sans secret : ports/chemins — mais mauvaise pratique, contredit .env.example)	.gitignore ; app/.env.dev, .preprod, .prod
Export PPT via execFile(python,[script,...]) SANS shell (pas d'injection) ; binaire piloté par env PYTHON non exposé à la requête	app/src/server.js:867,880

VSCode2 — Interview-to-Deck (FastAPI)

Dernier commit : 2026-07-24T06:06:13+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	 ok	31 fichier(s) de test, coverage configuré
Test fonct.	 ok	17 test(s) à vérification réelle
Revue code	 ok	hook pré-commit, bmad-code-review
Revue incr.	 ok	skill + hook SessionStart
Design	 ok	deck-design-review, deck-design-library
Doc	 ok	README+usage, wiki+html, CLAUDE.md
Cadrage produit	 moyen	persona, besoins
Pratiques+rules	 ok	linter, CI, CLAUDE.md, conventions
Sécu (proxy)	 ok	.env gitigné, deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ● ok

Commentaire : Gestion d'erreur solide et homogène (hiérarchie `AIError/TranscriptionError`, messages UI lisibles, `timeout+retry` ciblé, parsing JSON tolérant), le seul vrai trou étant l'absence de validation des uploads audio.

EXEMPLE CONSTATÉ	LOCALISATION
Upload audio sans validation de taille ni de type MIME : seul l'import .docx vérifie l'extension — DoS mémoire possible, garbage refusé seulement en aval	<code>app/routers/interviews.py:935,1382,951</code>
Les 'except: pass' comptés par ruff (S110) sont les warm-ups best-effort documentés du lifespan, pas des échecs silencieux réels	<code>app/main.py:54-59</code>
Bon point : garde-fou anti-path-traversal explicite au téléchargement, parsing JSON IA tolérant (fences, extraction de secours)	<code>app/routers/interviews.py:979-980</code> ; <code>app/services/ai_common.py:278-293</code>
Appels Ollama avec timeout configurable + relance ciblée sur timeout + message actionnable	<code>app/services/ai_common.py:340-401</code>

performance — ● moyen

Commentaire : De vraies optimisations existent (transcription parallèle multi-cœur, jobs de fond, chunking calibré), mais des endpoints `async` exécutent du travail CPU/IA synchrone qui bloque la boucle d'événements et les fichiers sont chargés entièrement en mémoire.

EXEMPLE CONSTATÉ	LOCALISATION
Endpoints 'async def' appelant du travail CPU-bound synchrone (Whisper, extraction IA) sans <code>to_thread</code> : bloque la boucle FastAPI pendant plusieurs minutes par requête	<code>app/routers/interviews.py:935+941,1382+1393,242+266</code>
'await file.read()' charge tout l'audio en mémoire sans cap ni streaming — un entretien de 1h30-3h passe en RAM (doublé au décodage PCM)	<code>app/routers/interviews.py:941,956,1393</code>
Appels IA en boucle séquentielle sur les tronçons (map-reduce non parallélisé)	<code>app/services/interview_libre_extract_ai.py:215-223</code>
Bon point : transcription découpée et parallélisée sur <code>ProcessPoolExecutor</code> (~1,8x) au-delà du seuil, jobs en <code>BackgroundTasks</code>	<code>app/services/audio_transcribe.py:161-179</code>

risque_technique — ● moyen

Commentaire : `pptx_deck` est une vraie bibliothèque de primitives partagée (importée par `pptx_export`, pas une duplication aveugle), mais dépendances non épinglées, gros module d'export et migrations SQLite artisanales pèsent sur la maintenabilité.

EXEMPLE CONSTATÉ	LOCALISATION
Toutes les dépendances non épinglées (>=) : fastapi, sqlalchemy, openai, faster-whisper, python-pptx... — build non reproductible	requirements.txt (toutes lignes)
pptx_export.py = 1636 lignes ; importe bien pptx_deck as D (pas de duplication massive) mais le helper 'budget_ok' est redéfini 3 fois	app/services/pptx_export.py:170,346,439
Migrations SQLite additives en f-string DDL : constantes internes (pas d'injection) mais mécanisme fragile (pas d'outil, pas de rollback)	app/db.py:53-85
B904 ruff : HTTPException levées dans des except sans 'from exc' (chaîne d'exception perdue au debug)	app/routers/interviews.py:980,983

securite — ● ok

Commentaire : Aucune clé en dur, .env gitigné, aucun vecteur d'injection dangereux (pas de shell=True/eval/pickle/yaml.load/os.system), ORM paramétré et fichiers uploadés renommés côté serveur.

EXEMPLE CONSTATÉ	LOCALISATION
Clés API strictement en variables d'environnement, .env.example en placeholders, .env gitigné	app/services/ai_common.py:35-43 ; .gitignore:8
Aucun appel dangereux : grep shell=True/eval/pickle/yaml.load/os.system => 0 résultat dans app/ et tools/	app/ (grep global)
Fichiers uploadés écrits sous un nom généré serveur (mission_id+timestamp+uuid), jamais le filename client ; téléchargement protégé du path-traversal	app/routers/interviews.py:963-964,979-980
Point mineur : absence de plafond de taille d'upload => risque théorique DoS mémoire plutôt que faille d'intégrité	app/routers/interviews.py:941,956

VSCode3 — Cadrage BMAD IAP (deck de synthèse)

Dernier commit : 2026-07-24T10:13:08+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	● moyen	3 fichier(s) de test, pas de coverage
Test fonct.	● ok	2 test(s) à vérification réelle

Revue code	● ok	hook pré-commit, bmad-code-review
Revue incr.	● ok	skill + hook SessionStart
Design	● moyen	deck-design-library, ppt-designer
Doc	● moyen	wiki+html, CLAUDE.md
Cadrage produit	● moyen	why
Pratiques+rules	● moyen	CLAUDE.md, conventions
Sécu (proxy)	● ok	deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ● moyen

Commentaire : La régression NameError du build() est bien réparée et l'échec réseau des photos a un repli propre, mais le template est chargé sans garde dès l'import, seul vrai point de fragilité restant.

EXEMPLE CONSTATÉ	LOCALISATION
Template chargé sans garde au niveau module (import) et dans new_prs() — un template-octo.pptx absent lève un FileNotFoundError brut ; importer generate_deck (ce que fait le test) suffit à casser	docs/cadrage- ppt/generate_deck.py:96,112
Régression historique corrigée : le build() v2.6 appelle uniquement des slide_* définies (plus de NameError du commit 26461d9 mentionné dans CLAUDE.md)	docs/cadrage- ppt/generate_deck.py:2624-2758
Bon réflexe anti-échec-silencieux sur les photos (try/except + repli nature_images + print) ; le repli du repli n'est pas gardé	docs/cadrage- ppt/generate_deck.py:287-298
Contenu 100% codé en dur (pas de source externe) : « données manquantes » ne s'applique pas ; géométrie vérifiée et remontée (exit 1)	docs/cadrage- ppt/generate_deck.py:2752-2758

performance — ● ok

Commentaire : Pas d'enjeu perf significatif (génération de deck statique) — seul coût réseau (photos Openverse) mis en cache disque.

EXEMPLE CONSTATÉ	LOCALISATION
Fetch réseau des photos mis en cache sur disque (génére une seule fois si _img/<scene>.png existe)	docs/cadrage- ppt/generate_deck.py:283-284

risque_technique — ● moyen

Commentaire : Générateur monolithique de 2767 lignes reposant sur des copies manuelles d'utilitaires de

projets frères (duplication à resynchroniser à la main), mais le chemin de génération est réellement couvert par un test fonctionnel qui build + rend.

EXEMPLE CONSTATÉ	LOCALISATION
generate_deck.py = monolithe de 2767 lignes, ~35 fonctions slide_* dans un seul fichier, sans découpage par chapitre	docs/cadrage-ppt/generate_deck.py (2767 l)
Duplication inter-projets par copie : pptx_deck.py copie du skill global, framed_image/nature_images/stock_images greffés via sys.path — resynchro manuelle, dérive possible	docs/cadrage-ppt/pptx_deck.py ; generate_deck.py:66-70
Bonne couverture du chemin de génération : test_generate_deck.py build() + 40 slides + géométrie + cadres photo + rendu réel LibreOffice→PDF	docs/cadrage-ppt/test_generate_deck.py:90-193
Code mort assumé : slide_sous_chapitre conservé sans appelant depuis v2.6 (documenté)	docs/cadrage-ppt/generate_deck.py:164

securite —  ok

Commentaire : Surface faible confirmée par grep : aucun shell=True/eval/pickle, subprocess uniquement en arguments-liste, aucun secret réel en dur, chemins dérivés de __file__ et de littéraux internes.

EXEMPLE CONSTATÉ	LOCALISATION
Aucun shell=True, eval(), ni pickle dans le périmètre — vérifié par grep	grep sur docs/cadrage-ppt, tests, scripts
subprocess uniquement en listes d'arguments (rendu LibreOffice du test, inventaire git) — pas d'injection	docs/cadrage-ppt/test_generate_deck.py:70-73
Aucun secret en dur : les occurrences « Secrets »/« credential » sont du contenu de slide (classification D4), pas des valeurs	docs/cadrage-ppt/generate_deck.py:658
Chemins non assainis mais non exposés : noms d'images dérivés de littéraux internes fixes et de __file__, pas d'entrée utilisateur	docs/cadrage-ppt/generate_deck.py:283

VSCode4 — Deck OHC RH dispositifs d'écoute (pré-code)

Dernier commit : 2026-07-24T10:05:49+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	 moyen	1 fichier(s) de test, pas de coverage

Test fonct.	● moyen	1 test(s) à vérification réelle
Revue code	● ok	hook pré-commit, bmad-code-review
Revue incr.	● ok	skill + hook SessionStart
Design	● ok	deck-design-review, deck-design-library, ppt-designer
Doc	● moyen	wiki+html, CLAUDE.md
Cadrage produit	● absent	aucun artefact de cadrage produit détecté
Pratiques+rules	● moyen	CLAUDE.md
Sécu (proxy)	● ok	.env gitigné, deny rules, guard git

Audit qualitatif du 2026-07-23 (lecture du code réel)

robustesse — ● moyen

Commentaire : Bon self-check bloquant et repli hors-ligne pour les photos Openverse, mais crashes durs non gardés sur template/média absents et quelques échecs « print-and-continue » silencieux.

EXEMPLE CONSTATÉ	LOCALISATION
Accès aux blobs média par clé sans garde : si la v6 (zip) n'a pas exactement les 5 image*.png attendues, KeyError brut et non explicite	<code>generate_deck_ohc.py:513-515,528 ;</code> <code>charger_media:85-92</code>
Aucune garde sur l'absence du template v6 : Presentation(SOURCE_V6)/charger_media lèvent FileNotFoundError non capturée si le fichier source manque	<code>generate_deck_ohc.py:1109-1110 ;</code> <code>SOURCE_V6:63-65</code>
Fetch Openverse correctement replié : try/except -> nature_images hors-ligne, timeouts 15s/20s côté stock_images — point fort	<code>generate_deck_ohc.py:271-279 ;</code> <code>stock_images.py:40,52</code>
Cadres photo vides gérés défensivement, mais l'image manquante ne fait PAS échouer build() — semi-silencieux (seul le test le rattrape)	<code>generate_deck_ohc.py:262-264 ;</code> <code>_repositionner_layout_chapitre:345-347</code>
Self-check géométrie + débordements texte BLOQUANT en fin de build (exit 1) — filet solide	<code>generate_deck_ohc.py:1143-1146 ;</code> <code>pptx_deck.verifier_geometrie:723</code>

performance — ● ok

Commentaire : I/O réseau et disque bien maîtrisés : photos Openverse mises en cache sur disque (fetch une seule fois), zip et template ouverts une seule fois par build.

EXEMPLE CONSTATÉ	LOCALISATION
------------------	--------------

Fetch réseau caché sur disque : `os.path.exists` court-circuite tout re-téléchargement — 4 requêtes à froid, 0 ensuite

`generate_deck_ohc.py:271-274`

Extraction zip de la v6 faite UNE fois (`charger_media` appelé une seule fois), pas par slide

`generate_deck_ohc.py:1109 ; charger_media:85-92`

4 fetch réseau séquentiels à froid — non parallélisés mais bornés et cachés, coût négligeable

`generate_deck_ohc.py:1119,1126,1132,1137`

risque_technique — ● moyen

Commentaire : `pptx_deck.py` est une copie explicite du module `VSCoDe2` (duplication cross-projet), avec un générateur de 1161 lignes truffé de magic numbers de layout et de noms de formes template fragiles, couvert par un seul fichier de test d'intégration.

EXEMPLE CONSTATÉ	LOCALISATION
Duplication assumée : <code>pptx_deck.py</code> (930 l) est une copie de <code>VSCoDe2</code> — risque de divergence entre les 3 copies, aucune source unique	<code>pptx_deck.py:7-8 (docstring)</code>
Couplage fragile aux noms exacts de formes/layouts du template ('Google Shape;201;p40', '51 - Chapitre [2]') : un changement de template casse en silence	<code>generate_deck_ohc.py:322-331,341-349 ; _layout:95-99</code>
Magic numbers de layout omniprésents ; ~8 cartes KPI dupliquées quasi à l'identique (fonctions slides très longues)	<code>generate_deck_ohc.py:519-622 (slide_personas ~100 l)</code>
Imports cross-skill via <code>sys.path.insert</code> (<code>framed_image/nature_images/stock_images</code> depuis <code>.claude/skills</code>) : couplage chemin-relatif au REPO	<code>generate_deck_ohc.py:54-58</code>
Couverture : ~40 assertions d'un seul fichier d'intégration pour ~2100 lignes ; état global mutable <code>_IDS_BLOB = iter(...)</code> (StopIteration possible)	<code>test_generate_deck_ohc.py (unique) ; generate_deck_ohc.py:148</code>

securite — ● ok

Commentaire : Aucun `eval/pickle`, `subprocess LibreOffice` en forme-liste sans `shell=True`, fetch réseau avec timeouts et slug assaini ; seul bémol, aucune allowlist de schéma ni plafond de taille sur l'URL image renvoyée par l'API.

EXEMPLE CONSTATÉ	LOCALISATION
<code>subprocess</code> (LibreOffice, test uniquement) en forme-liste avec <code>timeout=120</code> , SANS <code>shell=True</code> — pas d'injection	<code>test_generate_deck_ohc.py:67-71</code>
Aucun <code>eval</code> / <code>pickle</code> dans tout le périmètre scripts (grep vide)	<code>scripts/</code>

Téléchargement de l'URL image de l'API Openverse via urllib.urlopen sans allowlist de schéma ni plafond sur r.read() ; atténué par endpoint HTTPS de confiance et timeout 20s

stock_images.py:50-53

Écritures fichier bornées à des chemins fixes ; la requête utilisateur est assainie en [alnum/_] avant usage comme nom de fichier — pas de traversée

generate_deck_ohc.py:66-68, 241-242

VScode5 — Supervision multi-projets (ce projet)

Dernier commit : 2026-07-24T11:07:03+02:00

Dimensions mesurées (scan déterministe)

DIMENSION	NIVEAU	CONSTAT MESURÉ
Test tech.	orange moyen	2 fichier(s) de test, pas de coverage
Test fonct.	red absent	aucune vérif fonctionnelle réelle détectée
Revue code	orange moyen	bmad-code-review
Revue incr.	red absent	absente
Design	purple n/a	ne produit pas de deck
Doc	green ok	README+usage, wiki+html, CLAUDE.md
Cadrage produit	green ok	persona, why, besoins, valeur + brief BMAD
Pratiques+rules	orange moyen	linter, CLAUDE.md
Sécu (proxy)	green ok	deny rules, guard git

Audit qualitatif du 2026-07-24 (lecture du code réel)

robustesse — orange moyen

Commentaire : Défensif sur les I/O (gardes typées OSError/ValueError, subprocess bornés par timeout) mais échecs avalés sans trace et code du hub toujours sans test — inchangé par P1.

EXEMPLE CONSTATÉ	LOCALISATION
read_json/read_text avalent toute erreur en None sans trace — un JSON corrompu d'un projet passe inaperçu au lieu d'être signalé	scripts/scan_projets.py:read_json/read_text
log_usage (hook PostToolUse) fail-open sur except Exception - > exit 0 : correct pour ne pas casser le flux outil, mais masque	.claude/supervision/log_usage.py:41

toute erreur de journalisation d'usage

sync_dispositif.read_config() ouvre projets.json sans garde — projets.json absent/corrompu lève un traceback non intercepté (fail-loud acceptable pour un CLI, mais message brut)

`.claude/dispositif/sync_dispositif.py:48`

Toujours aucun test sur le code du hub (scan_projets, scan_transcripts, log_run, sync_dispositif) — une régression passe inaperçue (dette assumée, cf. CLAUDE.md)

`scripts/ + .claude/ (pas de tests/)`

performance — ● moyen

Commentaire : Coûts bornés (6 projets, plafond 4000 fichiers/projet) ; deux points à surveiller si la flotte grandit, inchangés par P1.

EXEMPLE CONSTATÉ	LOCALISATION
_walk_code lit le CONTENU intégral de chaque fichier de test pour chercher les marqueurs fonctionnels — I/O proportionnel au nombre de tests, non caché	<code>scripts/scan_projets.py:analyse_pratiques</code>
refresh_local_scans lance les scans en séquentiel (subprocess bloquant, timeout 90 s chacun) — parallélisable si la flotte grandit	<code>scripts/scan_projets.py:refresh_local_scans</code>

risque_technique — ● moyen

Commentaire : RÉSOLU — la dette critique (6 copies divergentes de scan_transcripts.py/log_run.py) est éteinte par un canon unique + sync_dispositif.py (flotte 12/12 à jour, 0 dérive) ; restent un scan_projets.py monolithique et l'absence de tests.

EXEMPLE CONSTATÉ	LOCALISATION
RÉSORBÉ : duplication cross-projet remplacée par un canon unique (.claude/dispositif/canon/) propagé par sync_dispositif.py avec en-tête « généré » ; un fix se fait une fois puis --sync. Le canon a réuni deux améliorations jusque-là éparpillées (skills-par-lecture ex-VSCo1 + arbitrage-par-catégorie ex-VSCo3)	<code>.claude/dispositif/</code> <code>(sync_dispositif.py --check : 12</code> <code>à jour, 0 dérive)</code>
scan_projets.py est un monolithe de 1660 lignes (collecte + rendu Markdown + rendu HTML + catalogue des pratiques + répertoire craft dans un seul fichier) — candidat à un découpage par responsabilité	<code>scripts/scan_projets.py</code>
Chemin critique (le dispositif de supervision entier) sans aucun test de non-régression — le canon partagé amplifie l'enjeu : un bug du canon se propage désormais aux 6 projets	<code>scripts/, .claude/supervision/,</code> <code>.claude/orchestration/,</code> <code>.claude/dispositif/</code>

securite — ● ok

Commentaire : Aucune surface d'injection : subprocess en arguments-liste (jamais shell=True), pas

d'eval/exec/pickle/yaml.load, pas de secret ni de .env dans le dépôt ; sync_dispositif n'écrit que des chemins dérivés de projets.json (config locale de confiance).

EXEMPLE CONSTATÉ	LOCALISATION
subprocess.run toujours en liste d'arguments ([sys.executable, ...], [git, ...]) — pas d'injection de commande	scripts/scan_projets.py:refresh_local_scans/git_last_commit
sync_dispositif écrit dans des chemins construits par os.path.join à partir de projets.json (non exposé) — pas d'entrée utilisateur externe ; sûr tant que projets.json reste une config locale	.claude/dispositif/sync_dispositif.py:write_crlf